

École nationale des ponts et chaussées

Projet de département IMI — 2025–2026

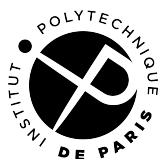
Du Structured Reinforcement Learning à un jeu Atari : étude, adaptation et benchmark

Yacine KLIKEL Arthur GORIN Léo CHENG
Matteo LE GOFF Neil BENMOUFFOK Pierre BRESSOLLETTE

Encadrant : Pierre-Cyril AUBIN

Rapport final — Juin 2026

Article de référence : Hoppe, Baty, Bouvier, Parmentier, Schiffer, *Structured Reinforcement Learning for Combinatorial Decision-Making*, NeurIPS 2025.



**INSTITUT
POLYTECHNIQUE
DE PARIS**

Code : ré-implémentation Python (PyTorch / Gymnasium) et Julia (Flux / ALE)

1 Introduction

1.1 Le cadre du Reinforcement Learning

Le but initial du projet était le suivant : *"Comprendre le nouveau paradigme du Structured Reinforcement Learning (SRL) présenté dans l'article [2], étudier les six problèmes de recherche opérationnelle (RO) présentés dans ce dernier, puis appliquer la méthode à différents jeux Atari."*

Présentons en premier lieu les bases théoriques du Reinforcement Learning (RL). Le RL est une branche du machine learning dans laquelle un agent autonome apprend à réaliser des successions d'actions, à partir d'expériences par essais-erreurs, de sorte à optimiser une récompense quantitative au cours du temps. L'agent interagit avec un environnement et prend ses décisions en fonction de son état courant. En retour, l'environnement procure à l'agent une récompense quantitative. L'agent cherche, au travers d'expériences itérées, un comportement décisionnel (appelé stratégie ou politique) optimal, en ce sens qu'il maximise la somme des récompenses au cours du temps. On adopte le formalisme de processus de décision markovien (MDP) défini de la manière suivante :

- Un ensemble fini $\mathcal{S}$ d'états que l'agent peut prendre.
- Un ensemble fini $\mathcal{A}(s)$ d'actions disponibles pour chaque état $s \in \mathcal{S}$.
- Un ensemble de valeurs scalaires "récompenses" $\mathcal{R}$ que l'agent peut obtenir.
- Un modèle stochastique de transition d'état $\mathbb{P}(s', r | s, a)$.

La recherche d'une politique optimale π^* est un problème d'optimisation complexe, on recourt à l'utilisation d'algorithmes pour y parvenir. Plusieurs grandes familles d'algorithmes existent [1], on les choisit selon des critères comme la structure de l'espace d'actions, l'accès à l'environnement ou encore le coût des interactions entre l'agent et l'environnement.

La *Q-valeur* $Q^\pi(s, a)$ mesure la qualité d'une action a prise dans un état s : c'est l'espérance du retour, c'est-à-dire de la somme des récompenses futures actualisées par un facteur $\gamma \in [0, 1]$, lorsqu'on joue a depuis s puis qu'on suit la politique π . Connaître la Q-valeur optimale suffit à agir de façon optimale : il suffit de choisir dans chaque état l'action de plus grande Q-valeur.

1.2 État de l'art et tractabilité sur de grands espaces d'actions

L'arrivée du deep RL a permis un cadre d'application plus vaste au RL : au lieu de tabuler une valeur pour chaque couple état/action $Q(s, a)$, on approxime Q par un réseau de neurones Q_θ . Cette méthode a l'avantage de pouvoir gérer des espaces d'états complexes (images, signaux ...) en apprenant une représentation utile du problème. L'état de l'art est alors passé d'un cadre d'application très restreint à des problèmes concrets comme Atari, Go, la robotique moderne et de nombreux autres.

Dans un cadre d'application à des problèmes industriels de RO (scheduling, vehicule routing ...), la structure de l'espace d'action est combinatoire : une action n'est plus une direction mais un objet structuré à part entière. De ce fait, un réseau de neurones classique ne peut pas produire une action structurée valide dans cet espace.

Les approches existantes butent toutes sur le même obstacle : l'espace d'action combinatoire est trop grand et trop structuré pour être traité directement. Les méthodes classiques de *policy gradient* (REINFORCE, PPO, SAC) peinent à y explorer efficacement. Les algorithmes *predict-and-optimize* séparent la prédiction de la décision, ce qui dégrade les performances en contexte dynamique. Une approche plus récente, les pipelines COAML (Combinatorial Optimization Augmented Machine Learning), intègrent directement un solveur combinatoire dans l'architecture du modèle, permettant un apprentissage bout-en-bout. Ces pipelines peuvent être entraînés par imitation à partir de solutions expertes, mais cette approche requiert des données hors ligne difficiles

à obtenir et ne permet pas à l'agent d'apprendre à partir de ses propres erreurs. Leur verrou central reste la non-différentiabilité du solveur pour la rétropropagation: son gradient est nul presque partout car l'espace d'actions est discret.

1.3 L'apport du SRL

Le SRL est le premier cadre d'entraînement d'une pipeline COAML de bout-en-bout par pur renforcement, sans politique experte.

Cette innovation repose sur deux mécanismes d'entraînement conjoints : une perturbation gaussienne des scores θ en sortie de réseau génère un ensemble d'actions candidates, que le critic évalue pour construire une cible $\hat{a}$ online. La Fenchel-Young loss permet ensuite de rétropropager à travers le solveur combinatoire en direction de cette cible, sans jamais nécessiter de démonstration experte. En outre, SRL ne suppose aucune structure particulière sur le critic ni sur le solveur combinatoire, ce qui le rend générique à une large classe de problèmes combinatoires.

1.4 Reformulation et objectifs

L'encadrant attendait deux étapes : Comprendre l'implémentation du SRL à travers l'étude des problèmes de RO de [1] (un problème par membre du groupe) et, une fois l'algorithme bien maîtrisé, tester son implémentation dans un autre environnement RL comme les jeux Atari.

Notre objectif a donc été d'implémenter l'algorithme SRL aux jeux Atari (un jeu par membre du groupe) pour tester la capacité d'adaptation de SRL dans des environnements RL pour lesquels il n'a pas été pensé initialement.

L'algorithme présente des libertés de choix et chaque membre du groupe a décidé de sa propre implémentation. Certaines ont donné des résultats performants et d'autres n'ont pas donné de résultats probants. Notre valeur ajoutée lors de ce projet réside donc dans l'analyse des résultats de nos différentes implémentations : nous identifions certaines conditions sous lesquelles SRL transfère bien hors de son cadre d'application original, et celles qui mènent à un échec d'apprentissage.

2 Cadre technique : du SRL aux jeux Atari

2.1 Le pipeline SRL en deux phases

On formalise chaque jeu comme un MDP : à chaque pas l'agent observe un état s , choisit une action $a \in \mathcal{A}(s)$, reçoit une récompense r et transite vers s' . Le SRL remplace l'acteur neuronal usuel par une chaîne *CO-augmented Machine Learning* (Figure 1) :

- **Inférence** (chemin noir) : un réseau d'encodage φ_w transforme l'état s en scores θ ; la CO-layer $f(\theta, s) = \arg \max_{\tilde{a} \in \mathcal{A}(s)} \langle \theta, \tilde{a} \rangle$ choisit l'action. La nature de φ_w dépend de la *représentation de l'état* : un CNN pour une entrée image, un MLP pour une entrée vectorielle (cf. §2.2).
- **Entraînement** (chemin bleu) : on perturbe θ par un bruit gaussien $\eta \sim \mathcal{N}(\theta, \sigma_b)$, on génère m actions candidates $a' = f(\eta, s)$, on les évalue par le critique Q_ψ , et on agrège par softmax en une action cible $\hat{a} = \text{softmax}_{a'}(\frac{1}{\tau} Q_\psi(s, a'))$. L'acteur est mis à jour en minimisant la perte de Fenchel-Young $\mathcal{L}_\Omega(\theta; \hat{a})$; le critique est mis à jour par TD-learning : on rapproche sa prédiction actuelle d'une cible composée de la récompense immédiate et de la valeur estimée de l'état suivant. Pour stabiliser l'apprentissage, cette valeur future est évaluée par un *critique cible*, dont les poids restent figés et ne recopient ceux du critique principal que tous les X pas de temps.

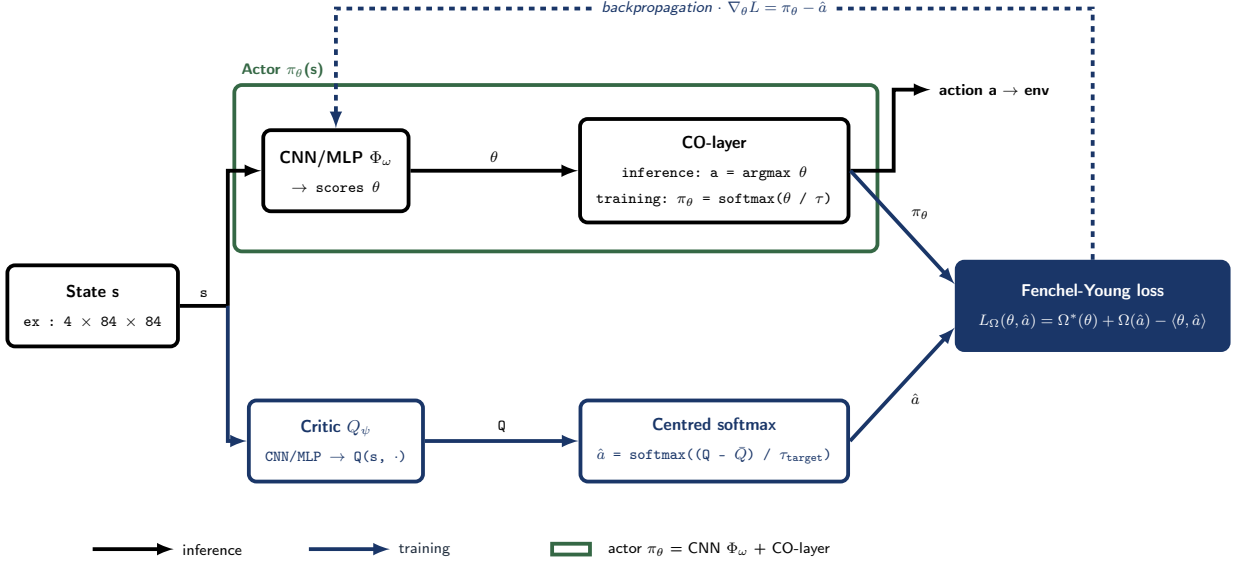


Figure 1: Architecture du SRL dans le cadre d'un jeu Atari

2.2 Deux approches de représentation de l'état

Pour s'appropriier le pipeline et le confronter à un éventail de jeux, le groupe a exploré deux familles de représentation de l'état Atari, partageant exactement la même mécanique SRL (CO-layer, critique, perte de Fenchel-Young) mais différant par l'encodeur φ_w :

(A) Approche pixels / CNN.

L'état est l'empilement de 4 frames grises 84×84 normalisées dans $[0, 1]$; φ_w est un réseau de neurones convolutionnel (CNN) (cf. §3.1). C'est la voie la plus proche des benchmarks classiques [3].

(B) Approche RAM / MLP.

L'état est le vecteur de la mémoire console (RAM Atari), normalisé ; φ_w est un perceptron multi-couche. Cette voie élimine la charge de la vision (pas de convolutions à apprendre) et raccourcit le temps de cycle, au prix d'une représentation plus opaque.

Couverture expérimentale. Au total, le groupe a abordé six jeux : *Breakout*, *Space Invaders*, *Q*bert*, *Beam Rider*, *Enduro* et *Pong*, répartis entre les deux approches. *Breakout* et *Enduro* constituent nos cas d'étude le plus abouti.

Pourquoi deux approches ? Comparer pixels et RAM permet de découpler ce qui relève de la perception (extraire un état utile) de ce qui relève de la décision (le cœur SRL).

Note d'implémentation : double portage Python / Julia. Le pipeline a été complètement codé en python par une partie du groupe et en parallèle en Julia par le reste d'entre nous. Si Julia offre théoriquement de meilleures performances de calcul brut, notre retour d'expérience est sans ambiguïté : l'écosystème Python (PyTorch + Gymnasium) reste nettement plus productif pour la conception et le débogage des réseaux, grâce à la maturité de ses bibliothèques (notamment sur MAC). Les résultats chiffrés présentés ici proviennent de la chaîne Python. Par exemple, l'approche avec CNN (avec donc un temps de calcul plus important) a uniquement été implémentée en Python.

2.3 Fonction de loss

2.3.A La perte de Fenchel-Young avec bruit gaussien

Dans un espace d'actions combinatoire, la CO-layer f est constante par morceaux par rapport à l'espace des actions, ce qui engendre des dérivées nulles presque partout et empêche la rétropropagation classique. Pour lisser cette fonction et permettre un apprentissage de bout en bout, l'article introduit une perturbation gaussienne $Z \sim \mathcal{N}(0, \epsilon)$. La perte de Fenchel-Young $\mathcal{L}_\Omega(\theta; \hat{a})$ est alors redéfinie en remplaçant la maximisation stricte par son espérance sous la perturbation aléatoire :

$$\mathcal{L}_\Omega(\theta; \hat{a}) = \mathbb{E} \left[\max_{a \in \mathcal{A}(s)} (\theta + Z)^\top a \right] - \theta^\top \hat{a} \quad (1)$$

Dans cette formulation, le terme d'espérance agit comme la conjuguée de Fenchel $\Omega^*(\theta)$ lissée par le bruit. La propriété fondamentale de cet artifice est de rendre la perte convexe et lisse par rapport aux scores θ générés par le réseau. Son gradient correspond exactement à la différence entre l'action espérée sous perturbation (notée π_θ) et l'action cible $\hat{a}$:

$$\nabla_\theta \mathcal{L}_\Omega(\theta; \hat{a}) = \pi_\theta - \hat{a} = \mathbb{E} \left[\arg \max_{a \in \mathcal{A}(s)} (\theta + Z)^\top a \right] - \hat{a} \quad (2)$$

En pratique, cette espérance est estimée par la méthode de Monte-Carlo à l'aide d'un estimateur de chemin (*pathwise estimator*). L'algorithme échantillonne plusieurs vecteurs de bruit, résout le problème combinatoire pour chacun via la CO-layer, puis moyenne les actions obtenues pour approcher π_θ . Le réseau de neurones peut ainsi être mis à jour par descente de gradient stochastique.

2.3.B La perte de Fenchel-Young avec régularisation entropique

La définition de la perte via une fonction de régularisation Ω offre une formulation alternative qui, pour un choix bien particulier de Ω , admet une *forme fermée* et évite tout échantillonnage. En prenant l'entropie négative pondérée par la température τ , $\Omega(p) = -\tau \sum_a p_a \log p_a$, dont la conjuguée de Fenchel est le *log-sum-exp* tempéré $\Omega^*(\theta) = \tau \log \sum_a e^{\theta_a/\tau}$, la perte de l'acteur s'écrit analytiquement :

$$\mathcal{L}_\Omega(\theta; \hat{a}) = \underbrace{\tau \log \sum_a e^{\theta_a/\tau}}_{\Omega^*(\theta)} - \underbrace{\tau \sum_a \hat{a}_a \log \hat{a}_a}_{\Omega(\hat{a})} - \underbrace{\sum_a \theta_a \hat{a}_a}_{\langle \theta, \hat{a} \rangle} \quad (3)$$

Son gradient est $\nabla_\theta \mathcal{L}_\Omega = \pi_\theta(\cdot | s) - \hat{a}$, avec $\pi_\theta = \text{softmax}(\theta/\tau)$. Il pousse la politique de l'acteur vers la cible $\hat{a}$ induite par le critique. Ce mécanisme est indépendant de la façon dont l'état est encodé : il s'applique identiquement aux deux approches du groupe. .

2.4 Premier point faible - Plus de *CO-layer*

La forme fermée (3) n'est calculable que parce que la somme $\sum_a$ porte sur une poignée de termes : c'est l'observation centrale du projet, commune aux deux approches. Là où les problèmes de RO connaissent une explosion combinatoire, un jeu Atari standard contrôle peu d'actions discrètes (p. ex. sur *Breakout* : LEFT, RIGHT, NOOP). Trois conséquences exactes en découlent, quel que soit l'encodeur (CNN ou MLP) :

1. **La CO-layer dégénère** : $\arg \max_a \langle \theta, a \rangle$ sur le simplexe revient à $\arg \max$ sur quelques scalaires. Aucun solveur combinatoire (Dijkstra, flot, PLNE...) n'est requis.
2. **La perturbation gaussienne devient superflue** : énumérer exhaustivement les actions remplace l'échantillonnage de m candidats dans un polytope. La cible est construite directement à partir des Q-valeurs : $\hat{a} = \text{softmax}((Q_\psi(s, \cdot) - \bar{Q})/\tau_{\text{target}})$.
3. **La perte de Fenchel–Young (3) se réduit à une entropie croisée softmax** entre les logits de l'acteur et la cible $\hat{a}$ de type *advantage*.

Conséquence : notre « SRL sur Atari » est, mathématiquement, un actor-critic softmax classique, que l'on encode l'état par CNN ou par MLP. Cela explique à la fois pourquoi il apprend (le mécanisme reste valide) et pourquoi il n'apporte pas beaucoup plus qu'une méthode non structurée : l'unique plus-value du SRL, gérer un espace d'actions combinatoire, disparaît avec le choix de l'environnement, et non avec celui de la représentation.

3 Résolution, difficultés et résultats

3.1 Architecture et données d'entraînement

Pour l'approche pixels, l'acteur π_θ et le critique Q_ψ partagent la même architecture CNN. Pour l'approche RAM, le CNN est remplacé par un MLP prenant le vecteur RAM en entrée ; le reste de la boucle est identique. Dans les deux cas l'acteur sort des logits bruts θ et le critique les Q-valeurs des actions. Un critique cible (copie figée, mise à jour périodiquement) stabilise les cibles TD : $y_t = r_t + \gamma \max_{a'} Q_{\bar{\psi}}(s_{t+1}, a')$, $\text{loss} (Q_\psi(s_t, a_t) - y_t)^2$. Les transitions sont stockées dans un replay buffer. C'est une mémoire tampon qui stocke les transitions vécues par l'agent ; au lieu d'apprendre sur la dernière transition observée, on échantillonne aléatoirement des mini-batches dans cette mémoire, ce qui brise la corrélation temporelle entre échantillons consécutifs. L'Annexe B donne la table complète des hyperparamètres retenus, selon chacun des jeux.

3.2 Points durs et choix d'implémentation assumés

L'adaptation du pipeline SRL à Atari nous a confrontés à plusieurs obstacles.

- **Temps de calcul important (inertie expérimentale)**. Un entraînement complet exige entre huit et quarante heures selon le jeu et la machine. Ce temps de cycle long restreint drastiquement le budget d'essais-erreurs, et impose une exploration empirique extrêmement ciblée. C'est en partie ce qui a motivé l'approche RAM : un encodeur plus léger pour raccourcir le cycle.
- **Réglage des hyperparamètres fin et fastidieux**. C'est de loin la partie où le plus de temps a été perdu. Chaque hyperparamètre devait être ajusté dans une fenêtre étroite, hors de laquelle l'agent échouait à apprendre quoi que ce soit d'utile :
 - **Exploration** : il fallait beaucoup d'exploration au début pour découvrir des politiques exploitables, mais pas trop non plus, sous peine de ne jamais converger vers une stratégie cohérente.
 - **Learning rate** : un taux mal calibré conduisait à un collage sur des politiques dégénérées — rester bloqué à gauche dans *Breakout*, ou foncer tout droit dans *Enduro* — l'agent figeant prématurément un comportement médiocre.
 - **Températures** ($\tau, \tau_{\text{target}}$) et autres paramètres de softmax/perturbation, dont le moindre

déséquilibre rendait la cible à soit trop molle (apprentissage lent), soit trop piquée (instabilité).

Couplée au temps de calcul ci-dessus, cette sensibilité a rendu chaque itération de réglage coûteuse, transformant la recherche d'hyperparamètres en goulot d'étranglement principal du projet.

- **Problèmes inhérents au modèle sur Atari.** La méthode plafonnait souvent, voici les pistes pouvant l'expliquer :

1. *Sparsity des rewards* : dans certains jeux (p. ex *Pong*, *Q*bert*), on a une récompense toutes les 50'000 frames, ce qui fait que le replay buffer est rempli de vide, empêchant l'acteur de trouver une policy à suivre.
2. *Diversité du replay buffer* : Une fois qu'une politique commençait à être suivie, l'agent s'y enfermait : le replay buffer se remplissait alors de transitions issues de cette seule politique, ce qui rendait difficile d'en sortir quel que soit le taux d'exploration (par exemple rester collé à un bord de l'écran.)
3. *Cible mouvante* : Avec une copie dure tous les X pas, le target cible saute brutalement à chaque mise à jour. Si l'intervalle est mal calibré, l'acteur poursuit une cible qui se déplace par à-coups, ce qui plafonne l'amélioration au lieu de la lisser. Réaliser une soft update (copier un certain pourcentage des poids à chaque pas) n'a pas amélioré la performance en général.

- **Pistes écartées et justification structurelle.** Nous avons délibérément laissé de côté les optimisations standard du RL moderne (Double-Q, Prioritized Experience Replay). Ayant démontré (cf. §2.4) que le SRL dégénère ici en un actor-critic classique, ces ajouts auraient vraisemblablement permis d'obtenir un meilleur score, mais sans apporter aucune information sur la viabilité du SRL en lui-même. Nous avons donc préféré concentrer nos efforts sur l'étude de la méthode SRL en elle-même, afin de valider ou non sa capacité à s'adapter à l'environnement Atari. Nous avons tout de même essayé de recréer une structure combinatoire comme nous allons le voir dans la partie suivante.

3.3 Tentative de restauration de la combinatoire : séquences d'actions

Puisque la dégénérescence vient de la taille de $\mathcal{A}(s)$, nous avons cherché à recréer une structure combinatoire. L'idée : au lieu de choisir une action unique par pas, le réseau prédit une *succession* d'actions sur plusieurs frames, ce qui fait croître exponentiellement la taille de l'espace de décision et redonne un rôle à la CO-layer. Cette approche a été testée de manière exploratoire sur des environnements comme *Beam Rider*, *Enduro* et *Q*bert*. Testée de manière exploratoire via des séquences d'actions basiques, cette approche n'a pas révélé d'amélioration manifeste. L'élargissement de l'espace d'actions semble introduire une certaine instabilité dans l'estimation des Q-valeurs en début d'apprentissage.

3.4 Résultats

Dans cette partie, nous présentons les résultats obtenus par notre modèle SRL sur deux jeux Atari. Cette section décrit la recherche, en pratique, de nos hyperparamètres et la façon dont se déroule l'apprentissage du modèle. Nous avons choisi de garder dans ce rapport uniquement l'exemple de deux jeux: un cas où l'implémentation est une réussite, et un cas d'échec, cependant les résultats, avec leur analyse, de chacun des six jeux traités est disponible en annexe C.

3.4.A Enduro

Dans le jeu *Enduro*, l'agent doit comprendre qu'il faut accélérer en permanence pour avancer, tout en évitant les autres voitures sur la route. Pour simplifier le problème sur ce jeu, nous avons réduit l'espace d'actions de l'agent à seulement 4 commandes utiles : avancer, avancer en tournant à gauche, avancer en tournant à droite, et freiner.

Pour trouver les bons réglages sans lancer de longs calculs inutiles, nous avons mis en place une méthodologie itérative pilotée par l'analyse des métriques internes d'entraînement.

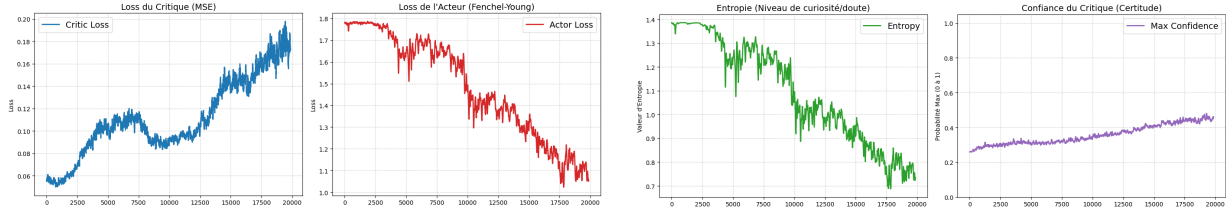


Figure 2: Tableau de bord de l'exploration lors d'un run court de réglage (20k étapes). Évolution des pertes (Critique et Acteur), de l'entropie et de la confiance maximale.

Un tableau de bord pour la sélection des hyperparamètres. Le point plus délicat a été le compromis exploration et exploitation. Plutôt que de lancer directement des entraînements de 10^6 étapes (1 jour de calcul), nous avons exploité les courbes de la Figure 2 sur des runs très courts d'environ 10 000 à 20 000 étapes (soit ~ 15 minutes de calcul).

Ces diagnostics rapides permettaient de scruter la "psychologie" de l'agent pour régler la température du softmax (τ), la variance du bruit d'exploration et le facteur pondérant l'entropie:

- **Le thermomètre de la curiosité (entropie/confiance) :** Au début, l'agent n'a aucune certitude et explore donc aléatoirement, n'ayant que 4 actions possibles, la probabilité qu'il en choisisse une est de $1/4$. Par la suite, ces deux métriques évoluent de manière quasi symétrique. Si l'entropie s'effondre trop vite et que la confiance sature prématurément vers 1,0, l'agent est devenu trop sûr de lui et s'enferme dans un optimum local (ex: refuser d'accélérer pour éviter tout risque de crash). Il faut alors augmenter τ ou le poids de l'entropie. À l'inverse, une entropie bloquée à un niveau élevé signale un agent papillonnant au hasard.
- **La dynamique révélatrice de la perte du Critique (courbe bleue) :** Contrairement à un apprentissage supervisé classique, une augmentation temporaire de cette perte n'est pas un signe d'échec, mais témoigne des paliers de découverte de l'agent.
 - D'abord, l'agent ne sait pas conduire : on prédit toujours un score nul, 0 dépassement.
 - Puis, l'agent avance tout droit ; on découvre les dépassements et surtout les crashes, le critique doit apprendre à prédire ces états
 - Ensuite, le critique comprend qu'en fonçant tout droit, l'agent va s'écraser, entraînant une baisse de l'erreur
 - Enfin, une seconde hausse lorsque l'agent apprend à esquiver, son travail devient bien plus fin, on n'évalue plus juste un unique comportement suicidaire

C'est seulement après avoir intégré cette nouvelle finesse d'anticipation, passé le cap des 50 000 étapes, que la perte du critique amorce sa véritable décroissance continue.

Le bon jeu d'hyperparamètres devait obtenir une entropie en baisse contrôlée, et une perte du critique stable après amorçage. Une fois ce comportement validé sur 15 minutes, l'entraînement était prolongé sur un test intermédiaire de 100 000 étapes pour confirmer la dynamique de convergence, avant de lancer le run final d'un million d'étapes.

Performances et efficacité d'échantillonnage. Une fois ce réglage validé, le modèle a été entraîné sur la durée complète. La Figure 3 illustre l'évolution du score au cours de cet apprentissage, et le comportement obtenu est particulièrement satisfaisant.

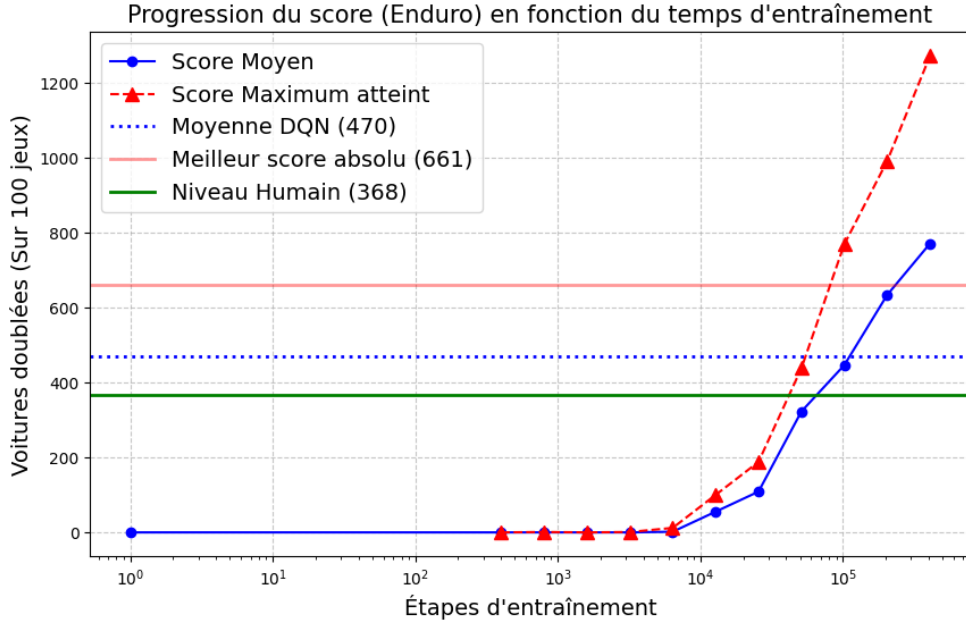
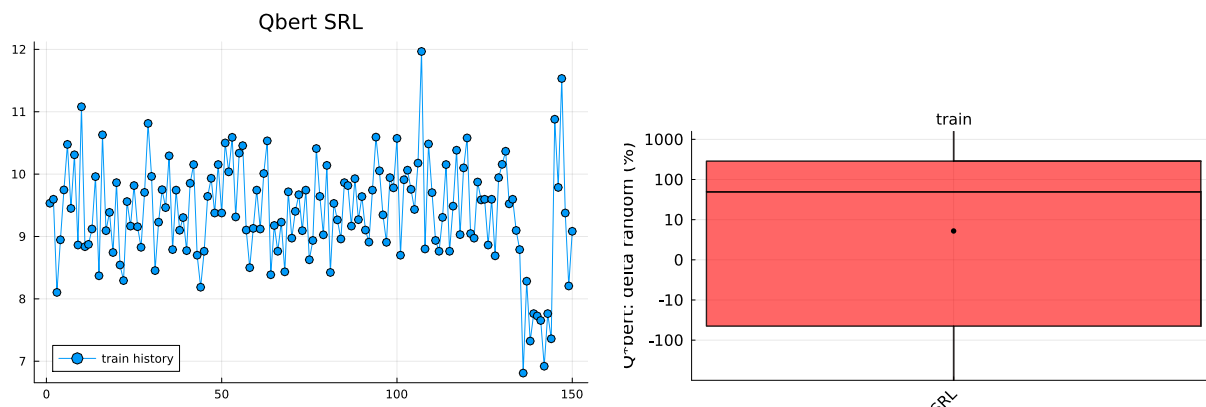


Figure 3: Progression du score sur le jeu *Enduro* en fonction du temps d’entraînement. Comparaison de l’agent SRL avec la baseline DQN. Les lignes en pointillés représentent les scores finaux atteints à l’issue d’un entraînement très long.

On y observe l’acquisition d’une politique de conduite très performante, où l’agent parvient à maintenir une accélération continue tout en esquivant le trafic. Au-delà du niveau absolu atteint, c’est l’efficacité d’échantillonnage de l’approche SRL qui est remarquable : nos scores finaux dépassent largement ceux de la référence historique DQN [3], et ce en nécessitant 50 fois moins d’étapes d’apprentissage. Cette convergence extrêmement rapide valide l’intégration de la couche de planification pour structurer l’exploration de l’agent.

3.4.B Q*bert

Dans le jeu *Q*bert*, l’agent évolue sur une pyramide de cubes ; son but est de changer la couleur du maximum de cases tout en évitant les ennemis et les chutes hors de la pyramide. L’espace d’actions de base compte 6 éléments (NOOP, FIRE, et les quatre directions correspondant aux sauts diagonaux sur la pyramide). Nous considérons des séquences de 2 actions consécutives, soit $6^2 = 36$ actions composites. La même méthodologie que pour *Enduro* a été employée pour la sélection des hyperparamètres. Cette fois, aucun jeu d’hyperparamètres n’a produit d’amélioration de performance convaincante au cours de l’entraînement : la courbe de récompense reste stable mais ne progresse pas. Le meilleur run obtient néanmoins une politique stable, sans effondrement sur 150 épisodes, dont le retour médian dépasse d’environ 50 % celui de la baseline aléatoire. Ce plateau est atteint dès la fin du *warmup*, et prolonger l’entraînement ne le modifie pas : sur des horizons plus longs (jusqu’à 500 épisodes), la performance n’augmente pas et tend même à se dégrader, sans jamais traduire d’apprentissage effectif.



(a) Récompense (moyenne sur 30 épisodes de test) au cours de l'entraînement. La performance se stabilise sans progresser. (b) Écart relatif au jeu aléatoire du modèle final ; le retour médian dépasse la baseline d'environ 50 %.

Figure 4: SRL sur Q^*bert : politique stable mais sans apprentissage, conservant un retour médian supérieur d'environ 50 % à la baseline aléatoire.

4 Conclusion, points acquis et perspectives

4.1 Ce qui a été établi

Le projet est parti d'un objectif d'application : « porter le SRL sur Atari » et a abouti sur le résultat suivant : la compréhension de ce que devient le SRL hors de son terrain de conception. Comme démontré mathématiquement en section 2.4, la méthode dégénère sur un espace d'actions aussi restreint pour se réduire à un actor-critic softmax classique. Loin d'être un échec, ce résultat éclaire la nature même de la méthode : sa plus-value réside majoritairement dans la structure combinatoire de l'espace d'actions, et non dans son appareillage d'entraînement.

4.2 Résultats les plus probants

Malgré cette dégénérescence théorique, le pipeline *apprend*, ce qui valide la cohérence de notre ré-implémentation (acteur, critique, perte de Fenchel–Young, replay, boucle d'entraînement). Les résultats se hiérarchisent clairement selon le jeu, et cette hiérarchie recoupe globalement celle des performances du DQN [3] : les jeux où nous obtenons un bon signal sont aussi ceux où le DQN excellait :

- **Enduro** : l'agent dépasse le DQN de référence, notre résultat le plus net.
- **Breakout** : l'agent atteint un niveau humain et *converge plus vite* que le DQN, illustrant le bénéfice de stabilité.
- **Cas plus mitigés** (Q^*bert , *Beam Rider*, *Pong*, *Space Invaders*) : signal d'apprentissage faible à nul, principalement dû à la rareté des récompenses et à l'absence de structure exploitable.

Que deux des six jeux atteignent ou dépassent une baseline DQN, sans aucune des optimisations modernes (Double-Q, PER), est en soi une confirmation de robustesse du cœur SRL.

4.3 Ce que cela apporte

Au-delà de la connaissance produite, l'intérêt pratique de ce travail est double. D'une part, nous livrons une ré-implémentation fonctionnelle d'une chaîne SRL ; c'est un point de départ réutilisable pour quiconque voudrait appliquer la méthode, notamment pour de futurs élèves IMI.

D'autre part, nous fournissons un diagnostic clair sur les performances du SRL dans des environnements de faible dimension. Le constat théorique dressé plus haut permet de faire gagner un temps de calcul (et donc un budget potentiel) précieux aux futurs utilisateurs. Ce travail confirme également que la méthode reste très compétitive face à l'algorithme de référence (DQN

[3]) sur certains jeux (*ex.* : *Breakout*, *Enduro*). Enfin, il permet d'émettre l'hypothèse forte qu'avec des choix d'architecture optimisés et une puissance de calcul suffisante, le SRL devrait pouvoir égaler cette même référence sur les environnements qui n'ont pas abouti ici (*ex.* : *Pong*, *Q*bert*, *Space Invaders*, *Beam Rider*).

4.4 Limites et points en suspens

Plusieurs limites circonscrivent la portée de nos conclusions :

- **Budget expérimental contraint.** Le coût de calcul (8 à 40 h par entraînement) a limité le nombre de seeds et l'ampleur du balayage d'hyperparamètres. Nos résultats sont donc indicatifs plus que statistiquement robustes, l'article original retient dix seeds par configuration, ce que nous n'avons pu reproduire.
- **Sensibilité aux hyperparamètres non maîtrisée.** La fenêtre de réglage viable (learning rate, températures, exploration) s'est révélée étroite et coûteuse à explorer ; nous ne pouvons exclure que de meilleurs réglages débloquent les jeux restés muets.
- **Tentative combinatoire non concluante.** L'idée de restaurer une structure via des séquences d'actions (§3.3) n'a pas, en l'état, produit de gain clair, et reste le principal chantier laissé ouvert.
- **Dépendance au critique.** Le signal d'apprentissage transite entièrement par la cible $\hat{a}$ induite par le critique ; un critique biaisé (sur-estimation, valeurs plates) plafonne mécaniquement l'acteur, sans que nous ayons pu le diagnostiquer finement.

4.5 Perspectives

Plusieurs pistes prolongeraient naturellement ce travail :

- **Augmenter nos capacités d'entraînement :** Un assouplissement des contraintes matérielles évoquées à la section précédente permettrait d'étendre nos tests sur les jeux à récompenses éparées et de valider des environnements plus vastes.
- **Aller au bout de la restauration combinatoire :** Les séquences d'actions sont la voie la plus prometteuse pour redonner un rôle à la CO-layer. Construire un espace de macro-actions structuré (avec un ordre d'exécution et des contraintes de transition) permettrait de tester réellement le SRL sur Atari. C'est, selon nous, la seule manière d'évaluer la méthode de façon équitable.
- **Valider sur un environnement à action réellement combinatoire :** Il serait intéressant de tester la méthode hors de la recherche opérationnelle, par exemple sur un jeu nécessitant des sélections multiples et simultanées. Cela permettrait de confirmer si les gains observés avec le SRL se transposent bien aux jeux vidéo, sans se limiter à des environnements basiques et déjà largement étudiés comme Atari. On peut par exemple penser aux échecs.
- **Réaliser un *benchmark* plus complet :** Il faudrait tester chaque choix de méthode sur l'ensemble des jeux afin de pouvoir totalement valider ou infirmer ces choix. De même, tester une gamme plus large d'hyperparamètres (qui seraient identiques sur chaque jeu pour avoir une réelle comparaison) serait très instructif.

En définitive, si l'application du SRL aux jeux Atari semble être un abus de puissance algorithmique face à la taille de l'espace d'actions, ce projet nous semble quand même être une réussite. Nous avons démontré qu'il était possible d'adapter, de stabiliser et de faire apprendre cette structure mathématique complexe très loin de ses cas d'usage industriels. Ce travail remplit donc pleinement son rôle de preuve de concept et de "crash-test" : il offre aux futurs utilisateurs une base de code fonctionnelle, tout en leur fournissant le recul nécessaire pour faire les bons choix. Le SRL a prouvé sa robustesse, et nous savons désormais qu'il s'agit d'une méthode efficace qu'il faut savoir garder en réserve pour les environnements où la complexité combinatoire justifie de libérer tout son potentiel.

Annexes

A Les six environnements de recherche opérationnelle de l'article

Cette annexe synthétise les six problèmes étudiés individuellement par les membres du groupe (travail de compréhension du rapport intermédiaire). Trois sont statiques, trois sont dynamiques.

Warcraft Shortest Paths (statique).

État : image de carte $96 \times 96 \times 3$ décomposée en grille 12×12 de coûts cachés. CNN $\rightarrow$ scores de cellules θ ; CO-layer = Dijkstra ; action = plus court chemin coin haut-gauche $\rightarrow$ bas-droite ; récompense = $-\text{coût total}$.

Single Machine Scheduling (statique).

Ordonnancement de tâches sur une machine ; CO-layer = problème d'ordonnancement linéaire en θ ; minimise un coût de retard pondéré.

Stochastic Vehicle Scheduling (statique).

Affectation de tournées à des véhicules sous aléa ; CO-layer = problème de flot/affectation ; minimise le coût opérationnel espéré.

Dynamic Vehicle Scheduling (dynamique, aléa exogène).

Requêtes spatio-temporelles révélées au fil du temps ; décider servir/reporter ; minimiser le coût de routage tout en servant tout.

Dynamic Assortment (dynamique, aléa endogène).

Choisir l'assortiment montré aux clients (choix selon un logit multinomial) ; les features d'items évoluent selon les décisions passées ; maximiser le revenu.

Gridworld Shortest Paths (dynamique, aléa endogène).

Trouver des chemins de coût minimal vers des cibles qui se déplacent une fois atteintes ; les coûts dépendent des chemins antérieurs.

B Hyperparamètres retenus

Jeu	Taille des couches	Taille du replay buffer	Learning rate	Écart-type du bruit gaussien
Breakout	Conv(32, 8×8 , /4) $\rightarrow$ Conv(64, 4×4 , /2) $\rightarrow$ Conv(64, 3×3 , /1) $\rightarrow$ Dense(512) $\rightarrow$ Dense($ A $)	2×10^5	Acteur : 5×10^{-5} Critique : 1×10^{-4}	(Pas de bruit gaussien) température softmax $\tau_{\text{acteur}} : 1.0 \rightarrow 0$ $\tau_{\text{target}} = 0.5$)
Space Invaders	résultat non communiqué			
Q*bert	(Entrée : RAM, 128 octets) Acteur : Dense(128 $\rightarrow$ 64) $\rightarrow$ Dense(64 $\rightarrow$ 6 ²) Critique : Dense(128 + 6 ² $\rightarrow$ 128) $\rightarrow$ Dense(128 $\rightarrow$ 64) $\rightarrow$ Dense(64 $\rightarrow$ 1)	8 000	Acteur : $1 \times 10^{-5} \rightarrow 1 \times 10^{-6}$ Critique : 1×10^{-3}	$\sigma_f = 1.0$ $\sigma_b : 1.0 \rightarrow 0.3$ $\tau : 1.5 \rightarrow 1.0$
Beam Rider	Conv(32, 7×7 , /2) $\rightarrow$ Conv(32, 7×7 , /2) $\rightarrow$ Conv(64, 5×5 , /2) $\rightarrow$ Conv(64, 5×5 , /2) $\rightarrow$ Dense($ A ^6$)	3×10^6	Acteur : 1×10^{-4} Critique : 2×10^{-4}	$\sigma_f = 0.1$ $\sigma_b = 0.05$
Enduro	cf partie			
Pong	MLP (512, 512, 3)	50 000	Acteur : 3×10^{-4} Critique : 3×10^{-4}	$\sigma_f = 0.3$ $\sigma_b = 0.1$

Table 1: Hyperparamètres choisis pour chaque environnement

C Résultats des jeux restants

C.1 Breakout

Breakout constitue, avec *Enduro*, notre cas d'étude le plus abouti. L'analyse des courbes met en évidence un apprentissage réussi, en plusieurs temps :

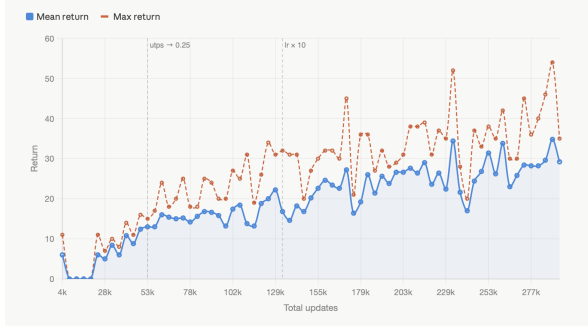


Figure 5: Apprentissage SRL sur *Breakout* : retour moyen (bleu) et maximal (rouge) par épisode en fonction du nombre total de mises à jour.

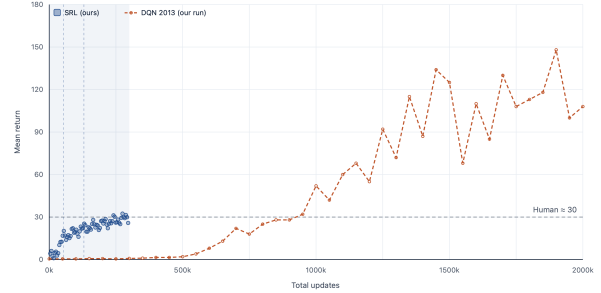


Figure 6: Comparaison de la vitesse de convergence : SRL (bleu) contre DQN 2013 [3] (rouge, approximatif)

- **Phase d'amorçage puis montée régulière** : après un démarrage hésitant (retour nul sur les ~ 20 premiers milliers de mises à jour, le temps que le critique se structure), le retour moyen croît de manière soutenue jusqu'à se stabiliser autour de 25–30, le retour maximal dépassant ponctuellement 50 (Figure 5). L'écart persistant entre retour moyen et maximal traduit une politique compétente mais encore bruitée, typique d'une exploration softmax non annihilée en fin d'entraînement.
- **Effet des interventions** : l'abaissement du ratio de mise à jour (0,25) puis l'augmentation du learning rate ($\times 10$) accompagnent la poursuite de la progression, sans déclencher l'effondrement observé sur d'autres jeux, c'est le signe d'une dynamique d'apprentissage robuste sur cet environnement. Sans ces interventions, l'agent ne dépassait pas 12 puis 20 reward moyen aux étapes.

Le point marquant n'est pas le niveau final atteint mais la *rapidité* avec laquelle il l'est. La Figure 6 compare notre pipeline au DQN de référence (2013) [3] : le SRL atteint le niveau humain (~ 30) en environ 290k mises à jour (environ 4-5h de training), là où le DQN requiert de l'ordre de 1 M de mises à jour pour franchir ce même seuil (20h à l'époque), soit un facteur ~ 3 en pas de gradient. Le DQN finit certes par atteindre des scores bien supérieurs (~ 150) sur un temps beaucoup plus long (plus de 50h).

C.2 Pong

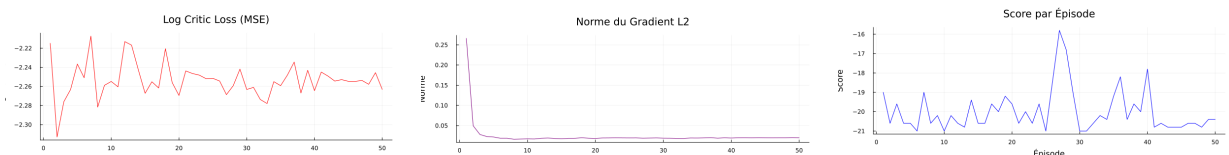


Figure 7: Évolution des métriques internes sur les 50 premiers épisodes. De gauche à droite : Perte du critique (MSE), norme du gradient L2 et score par épisode.

L'analyse des courbes met en évidence un échec de l'apprentissage dans cette configuration, caractérisé par un effondrement rapide de la politique :

- **Divergence du critique et stagnation du score** : La perte d'erreur quadratique (MSE) du critique oscille de manière chaotique, empêchant toute estimation fiable. Par conséquent, les scores stagnent autour de -21 (défaite totale), les légères variations n'étant que du bruit statistique.
- **Disparition du gradient** : La norme L_2 du gradient chute drastiquement et stagne à un niveau proche de zéro, bloquant la mise à jour des poids du réseau.

En résumé, faute d'un signal de récompense suffisamment dense pour lancer correctement la descente de gradient, l'agent s'effondre dans une politique dont il n'arrive plus à s'échapper.

Nota Bene : Bien que l'entraînement complet ait été mené sur 300 épisodes, la Figure 7 détaille l'évolution des métriques internes sur les 50 premiers (seuls les scores globaux ayant été tracés par la suite). Cette fenêtre initiale est néanmoins amplement suffisante pour diagnostiquer le comportement du modèle. Il n'y a eu aucune amélioration par la suite.

C.3 Beam Rider

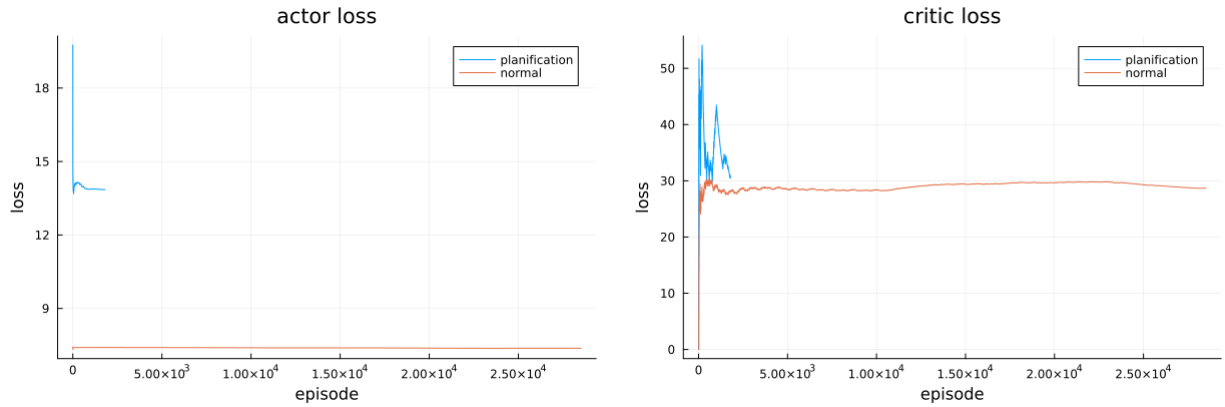


Figure 8: Évolution de la loss des acteurs et critique. En bleu, la loss d'un modèle qui planifie sur un horizon de 6 actions. En rouge, la loss d'un modèle qui ne choisit qu'une action par pas de temps.

Pour le jeu Beam Rider, il a été testé deux modes d'inférence différentes : un modèle simple avec un coup de par pas de temps et un modèle structuré qui planifie 6 actions par pas de temps. Sur la figure 8, on peut observer l'évolution des loss des deux modèles. Le modèle simple a un temps de calcul très faible devant le second modèle, ce qui a permis d'exécuter l'entraînement sur un plus grand nombre d'épisodes par rapport au second modèle, bien que le nombre total d'épisodes reste bien inférieur aux modèles de référence comme DQN [3]. On observe que la loss des critiques sont similaires, même si celle du modèle planifié est plus bruitée. En effet, l'architecture des deux critiques est identique et les épisodes explorés sont identiques, au moins sur les phases d'exploration. Quant aux deux acteurs, on observe que la loss du modèle avec planification se stabilise autour de 14, tandis que la loss du second modèle se stabilise autour de 7.

Pour le modèle normal, on observe que sur la figure 9 les rewards forment un plateau autour de 200, ce qui témoigne de la nécessité d'entraînement supplémentaire pour sortir de cet extremum local. Quant au modèle structuré, le début de la courbe semble indiquer de meilleures performances que le modèle simple, mais il est impossible de déterminer à ce niveau d'entraînement si cette supériorité n'est que la conséquence d'un bruit aléatoire.

Pour le modèle normal, on observe que sur la figure 9 les rewards forment un plateau autour de 200, ce qui témoigne de la nécessité d'entraînement supplémentaire pour sortir de cet extremum local. Quant au modèle structuré, le début de la courbe semble indiquer de meilleures performances que le modèle simple, mais il est impossible de déterminer à ce niveau d'entraînement si cette supériorité n'est que la conséquence d'un bruit aléatoire.

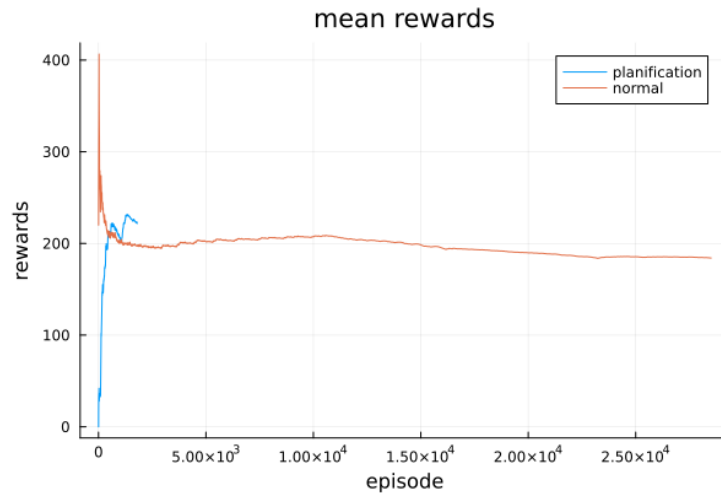


Figure 9: Évolution des rewards moyen. En bleu, la loss d’un modèle qui planifie sur un horizon de 6 actions. En rouge, la loss d’un modèle qui ne choisit qu’une action par pas de temps.

Références

- [1] Kai ARULKUMARAN et al. “A Brief Survey of Deep Reinforcement Learning”. In : *IEEE Signal Processing Magazine* 34.6 (2017). arXiv:1708.05866, p. 26-38.
- [2] Heiko HOPPE et al. “Structured Reinforcement Learning for Combinatorial Decision-Making”. In : *The Thirty-ninth Annual Conference on Neural Information Processing Systems*. 2026. URL : <https://openreview.net/forum?id=GS9o7u5njS>.
- [3] Volodymyr MNIH et al. “Playing atari with deep reinforcement learning”. In : *arXiv preprint arXiv:1312.5602* (2013).